LiveRecall

Ow Jun Duan (A0308148A) · Ng Yu Hao Benedict (A0299879N) · Max Khoo Zheng Ning (A0307430R) · Tan Yi Qian (A0301651U)

Abstract

LiveRecall is a browser-based classroom companion that distributes revision-based captions to concurrent readers and restores missed finalised text after interruptions. It combines monotonic revision state, local 15/30/60-second recall, bounded history and independent adaptive delivery channels. In the latest three-run Ubuntu results, 100 normal clients achieved 28.9ms P95 caption age with 100% final completeness. Under 10% slow readers, adaptive delivery reduced overall P95 from 2,254.2 to 27.0ms versus coalescing, while raising final completeness from 95.13% to 100%. All reported T1 – T5 workloads achieved 100% final completeness, combined delay/loss reduced fresh-final delivery to 89.96%, and adaptive scheduling increased latency under the 2 Mbit/s profile.

1 Motivation and Requirements

Students who briefly lose track of a lecture need the latest words and the preceding minute, not an unbounded transcript. A caption may change several times before becoming final. The main networking problem is therefore to distribute successive revisions, preserve final content, isolate slow receivers, and recover after reconnecting without replaying old or duplicate state. The deterministic **caption_source.py** is used, speech recognition is not part of the mandatory path.

2 Architecture and Application Protocol

END-TO-END DATA FLOW

SOURCE → SERVER → CLIENT
caption_source.py → SOURCE_HELLO / CAPTION_UPDATE → LiveRecallHub → independent ClientChannel writers → browser CaptionStore → transcript and local recall

The asynchronous Python WebSocket server separates source ingestion, authoritative per-session state, and outbound client delivery. After SOURCE_HELLO, the source emits CAPTION_UPDATE with session, segment, revision, source_ts, final and text. A student or load client sends CLIENT_HELLO (client_id, session, last_segment, last_revision, history_seconds), receives an ordered HISTORY snapshot of missing finals, and then consumes live CAPTION_UPDATE frames.

The LiveRecall hub supports consecutive deterministic caption-source runs without restarting the server or disconnecting existing clients. For a new source connection beginning at segment 1, the hub computes a segment offset using the session's highest accepted segment identifier. The normalized identifiers are then used for revision validation, state retention and client broadcast. This preserves existing client recovery cursors and prevents segment-ID collisions across sequential source runs.

Protocol frame	**Direction**	**Role**
SOURCE_HELLO	Source → server	Select caption session (e.g., L1)
CAPTION_UPDATE	Source → server → clients	Revision and durable final-state propagation
CLIENT_HELLO	Client → server	Join or resume using final-state cursor
HISTORY	Server → client	Only recently finalised state not already held

Implementation: solution_server.py → liverecall/hub.py, caption_state.py, client_channel.py, flow_control.py, semantic_queue.py, browser modules reside in client/. Required fields follow the assignment compatibility interface.

3 Revision Correctness, Local Recall and Recovery

3.1 Monotonic and Final-State Semantics

CaptionState accepts a newer revision for an active segment and rejects stale/equal revisions. Once `final=true` is accepted, subsequent updates for that segment are rejected. The browser CaptionStore applies the same checks and renders one row per segment, replacing provisional text in place rather than appending duplicates. Source revisions remain visible under normal conditions, and a final supersedes all pending provisional state for that segment.

Across sequential source runs: LiveRecallHub retains the highest accepted segment ID per session and fixes one offset for the new source connection when its first valid caption begins at segment 1. Updates are re-encoded with the adjusted segment ID before acceptance and broadcast. Final-state rejection and client cursors remain intact, simply clearing finalised state would incorrectly permit duplicates. This mechanism is intended for sequential, not concurrent, source runs.

3.2 Bounded State, Reconnect and Late Join

The server retains recent finalised captions for 90 seconds, capped at 4,096 items, and at most 64 active provisional segments. Each browser retains up to 90 seconds / 1,000 segments for local recall. “What did I miss?” initially shows 30 seconds and offers 15 and 60 seconds without requesting the server during normal operation.

After a disconnect, the browser automatically reconnects and resends its most recent finalised (segment, revision) cursor, the server returns newer finalised items within the requested history window (maximum 60 seconds). A new client uses cursor (0, 0). HISTORY and live updates merge into the same deduplicating store. The channel is registered before yielding after the history snapshot, avoiding a history/live handoff gap.

CORRECTNESS INVARIANTS
No older revision overwrites a newer one, no finalised segment is overwritten or displayed twice. HISTORY contains finalised entries only, ordered by segment, recovery is bounded by retained history. Consecutive source rounds must not reuse server-visible segment IDs.

4 Backpressure and Overload Strategy

The submitted `--policy` adaptive design decouples source intake from each client: the hub offers each accepted update to that client’s semantic queue, while an independent writer task sends to its own WebSocket. A blocked sender cannot stall ingestion or other writers. During NORMAL operation, the queue preserves useful revisions. Lag is inferred from an unanswered WebSocket ping exceeding minimum observed RTT by 0.3s, or from a transport write backlog above 64 KiB.

On congestion, the controller drains outstanding transport backlog, collapses superseded provisional revisions to the newest per segment, prioritises final captions when drain is clear, holds provisional-only state until it has waited at least 1s, and paces traffic starting at 0.5s per frame (backing off up to 4s). It probes for recovery and returns to NORMAL after lag subsides. The goal is durable final state with less network capacity spent on obsolete provisional revisions.

Bound / fail-safe	Default	Consequence
Pending segments per client	256	Overflow closes WebSocket 1013; client reconnects for HISTORY
Pending revisions per segment	16	Oldest provisional may be removed at cap
Delivered-state memory per client	512 segments	Bounded deduplication record
History / active / sessions	4,096 / 64 / 16	Time and count caps constrain application state

Comparison controls: FIFO intentionally queues every revision without a bound, coalesce collapses pending superseded revisions but lacks adaptive lag detection. FIFO is an experimental baseline, not the submitted default.

5 Public Experiments and Measurement Method

Measurements were obtained on Ubuntu using the unchanged deterministic source and public load-test analyser. T1 used 20 clients/20s, T2 used 100 clients/40s, T3 used 100 clients/60s including 10% readers delayed by 500ms, T4 used 100 clients/60s including 20% disconnected after 15s for 5s, and T5 used 100 clients/40s with tc/netem or tbf on loopback (lo). Each scenario was run independently three times. Every displayed value is the median of three per-run statistics, not a pooled percentile.

Caption age is client receive time minus source_ts, P50/P95/P99 characterise the distribution among accepted live updates and exclude browser paint time. “Fresh” means accepted live messages within 500ms, fresh-final uses first correct delivery of each generated final, including recovered finals. Completeness counts unique finalised segments received over all expected client–segment pairs. Coverage counts unique live revisions against generated revision–client pairs.

5.1 Required T1–T5 Results

Workload	P50 ms	P95 ms	P99 ms	Fresh	Fresh final	Final	Coverage
T1 · basic (20)	6.3	17.4	24.8	100.00%	100.00%	100.00%	100.00%
T2 · scale (100)	14.6	28.9	44.4	100.00%	100.00%	100.00%	100.00%
T3 · 10% slow	14.8	31.0	6,089.7	96.93%	94.21%	100.00%	94.39%
T4 · reconnect	13.0	24.3	30.6	100.00%	98.42%	100.00%	98.42%
T5 · 2 Mbit/s	27.0	51.6	65.0	100.00%	100.00%	100.00%	100.00%
T5 · delay	207.3	254.9	282.8	100.00%	100.00%	100.00%	100.00%
T5 · loss	15.4	28.6	216.6	99.99%	99.94%	100.00%	98.87%
T5 · delay + loss	263.3	705.2	1,036.9	87.54%	89.96%	100.00%	63.20%
T5 · reorder	61.4	73.0	79.6	100.00%	100.00%	100.00%	100.00%

Table 1. Three-run median client-receive caption ages (ms) and delivery ratios from the latest summary. All 15 reported scenarios recorded zero observed duplicate and stale messages. Coverage is unique live revisions divided by expected source revision–client pairs.

5.2 Interpretation: Scale, Slow Readers and Recovery

T2 achieved 100% final completeness and full revision coverage at P95 28.9ms. In T3, all expected finals arrived despite 94.39% live revision coverage: normal-client P95 was 25.4ms versus 8,147.3ms for slow clients, driving overall P99 to 6,089.7ms. T4 recovered the disconnected population with a 9.1ms recovery P95 and 100% final completeness, with zero observed duplicates or stale messages. Fresh-final remained 98.42% because some recovered finals can arrive correctly but after the 500ms freshness threshold.

5.3 Linux Impairment Behaviour

Captured qdiscs apply 2 Mbit/s TBF (4 KiB burst, 400ms latency), 100 ± 20ms delay, 3% loss, combined 120 ± 30ms delay/3% loss and 20ms reordering. Under 2 Mbit/s shaping, P95 was 51.6ms with 100% final completeness and revision coverage. With 3% loss alone, P95 remained 28.6ms but P99 rose to 216.6ms, illustrating a long retransmission-related tail not visible in P95. Combined delay/loss yielded P95 705.2ms and P99 1,036.9ms, 87.54% fresh live updates, 89.96% fresh finals and 63.20% revision coverage, while final completeness remained 100%. TCP/WebSocket reliability did not guarantee timely revisions.

6. Before/after optimisation evaluation

The public slow-reader workload was repeated under three --policy settings: FIFO (basic comparator), coalesce (revision suppression without adaptive lag control), and adaptive (submitted design). Tables report medians over three independent runs per mode. Workload settings were matched, without a tested-commit fingerprint embedded in historical outputs, attribution beyond these controlled comparisons is limited.

6.1 T3: Slow-reader Stress (100 clients, 10 slow)

Mode	P50 ms	P95 ms	P99 ms	Fresh final	Final comp.	Normal P95	Slow P95
FIFO	10.7	2,254.3	24,011.7	90.00%	95.13%	19.5	28,057.7
Coalesce	11.3	2,254.2	24,007.6	90.00%	95.13%	18.2	28,064.7
Adaptive	14.2	27.0	6,114.3	94.08%	100.00%	24.0	8,141.8

Table 2. Overall caption-age P50/P95/P99, and normal- and slow-client-specific P95. Final completeness uses expected client–final pairs.

Relative to coalescing, adaptive reduced overall P95 by **98.80%** (2,254.2 to 27.0ms), P99 by 74.53% (24,007.6 to 6,114.3 ms) and slow-reader P95 by 70.99% (28,064.7 to 8,141.8ms). Final completeness rose from 95.13% to 100%, and fresh-final delivery rose from 90.00% to 94.08%. This gain came with lower revision coverage (94.39% versus 95.18%) and higher normal-client P95 (24.0 versus 18.2ms). Adaptive substantially improved tail behaviour without eliminating multi-second slow-client delays.

6.2 T5: 2 Mbit/s Constrained-bandwidth Comparison

Mode	P50 ms	P95 ms	P99 ms	Fresh final	Final comp.	Update cover.
FIFO	15.9	39.2	44.0	100.00%	100.00%	100.00%
Coalesce	16.5	39.9	43.7	100.00%	100.00%	100.00%
Adaptive	27.3	51.7	58.9	100.00%	100.00%	100.00%

Table 3. Three-run medians from the 100-client, 40-second matched-policy comparison under 2 Mbit/s TBF shaping.

Adaptive P95 (51.7ms) was **31.89% higher** than FIFO (39.2ms), although all three policies achieved 100% freshness, revision coverage and final completeness. The tested 2 Mbit/s limit was not restrictive enough for semantic suppression to offset adaptive probing and pacing overhead. Adaptive backpressure helps when receivers actually fall behind, it is not inherently faster under every network. More restrictive bandwidth and burstier sources could reveal the crossover point.

OPTIMISATION FINDING
Adaptive preserves finals and sharply improves receiver-backpressure latency but adds measurable overhead at the tested 2 Mbit/s setting. Both favourable and unfavourable results are reported.

7 Failures, Trade-offs and Limitations

7.1 What the Experiments Did Not Solve

Slow readers still experienced substantial latency under adaptive mode (comparison slow P95 8.14 s, overall P99 6.11 s). Under combined delay and loss, all finals arrived but only 89.96% within the 500ms freshness threshold, revision coverage fell to 63.20%, and overall P99 exceeded 1s. Adaptive scheduling adds overhead without congestion. At the 256-segment pending cap, close code 1013 bounds application backlog, but reconnect recovery remains limited by retained history.

7.2 Validity and Resource Bounds

The source and population are deterministic, but latency is measured at simulated Python clients, not browser paint. Tests exclude speech recognition, classroom Wi-Fi contention, mobile power management and clock-offset effects and use at most 100 clients. A five-snapshot T3 monitor saw 10 congested clients, 390 cumulative coalesced updates and RSS of 32.8–33.4 MiB over 20s: short-term stability, not a full-run bound. Queue high-water marks, CPU usage and post-overload convergence remain less thoroughly characterised.

The sequential-source regression was reported as passing during Windows validation on 4 November 2026, against commit 51b0be7 with local modifications to `liverecall/hub.py`. Two sequential source runs produced finalized segment IDs 1 – 3 and 4 – 6, and reconnecting from cursor 3 recovered segments 4–6. This validates the tested working-tree implementation; the submitted commit should be checked separately to establish identical behaviour.

7.3 Engineering Trade-offs

The design gives normal users full revision visibility, when possible, then applies semantic reduction when lag is detected. Independent channels isolate healthy readers, and final priority favours visible correctness. Ping-based detection adds control traffic, drain/pacing delays some provisional text, bounded history discards older context. Intentionally unbounded FIFO illustrates why “queue every update” is not the submitted overload policy, even when no-contention P95 is lower.

8 Future Work / FYP Direction

A future FYP could study capacity-aware semantic fan-out for lecture captioning. We would instrument queue peaks, RSS, ping lag and policy transitions, evaluate 500+ clients, synchronised reconnects and bandwidth profiles below 2 Mbit/s, and optimise the transition between full revision delivery and final-priority operation. A classroom prototype could integrate local real-time ASR via the existing protocol and measure how quickly students recover missed concepts, beyond networking-level freshness.

9 Conclusion

LiveRecall provides a revision-correct caption workflow, local 15/30/60-second recall, bounded state and incremental WebSocket recovery. In the latest public T1–T5 results, every scenario achieved 100% final completeness, T2 P95 was 28.9ms. Adaptive delivery reduced **overall T3 P95 under slow-reader stress** by 98.80% versus coalescing while retaining all finals. Persistent slow-client tails, worsening freshness under combined delay/loss and 2 Mbit/s overhead show that semantic backpressure must be assessed together with coverage and workload conditions. A separate repeat-source patch addresses consecutive deterministic runs, but this test summary alone does not verify it.

Appendix A. Reproducibility and Source Traceability

A.1 Environment and Installation

Experiments were run on Ubuntu Linux with Python 3.12, deterministic caption_source.py and simulated WebSocket clients. Server and load clients ran inside the same Ubuntu environment, tc/netem or tbf impaired loopback (lo). The latest user-supplied three-run SUMMARY.md does not include a tested-commit hash, so the new measurements cannot be attributed to a particular commit or to the sequential-source patch without additional provenance.

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip iproute2 git
git clone https://github.com/idunruniroll/CS3103.git && cd CS3103
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Launch the submitted server independently using `python solution_server.py --port 8765`. For automated public experiments, leave this server stopped: run_demo.py starts and stops its own instance. A left-over instance causes an address-already-in-use error.

A.2 Public tests and policy comparisons

From the repository root, the following scripts reproduce the public workloads. Each scenario and policy mode was repeated three times. Outputs are stored in experiments/data/.

```
RUNS=3 bash experiments/run_public_tests.sh       # T1–T4
RUNS=3 bash experiments/run_netem.sh             # T5 sudo/tc required
RUNS=3 WITH_BANDWIDTH=1 bash experiments/compare_policies.sh
python experiments/summarise.py experiments/data --output experiments/data/SUMMARY.md
```

T1: 20 clients/20s. T2: 100 clients/40s. T3: 100 clients/60s with 10% slow readers (500ms read delay). T4: 100 clients/60s, 20% disconnecting at 15s for 5s. T5: 100 clients/40s per impairment profile. Values are three-run medians, not pooled percentiles.

A.3 Captured network impairment profiles

Profile	Linux tc configuration on loopback (lo)
bandwidth_2mbit	tbf: 2 Mbit/s, 4 KiB burst, 400ms latency
delay_100ms	netem: 100ms delay, 20ms jitter
loss_3pct	netem: 3% loss
delay_loss	netem: 120ms delay, 30ms jitter, 3% loss
reorder	netem: 20ms delay, 20% reorder, 50% correlation, gap 1

Impairments were reset between profiles. To confirm the current state after testing, run `sudo tc qdisc show dev lo`. Baseline output may show qdisc noqueue after loopback shaping is removed.

A.4 Metric verification and source provenance

The public analyser reports caption age (client receive time minus source_ts), P50/P95/P99, 500ms live freshness, generated-final freshness, unique final completeness and live revision coverage. T3 adds normal/slow client latency, T4 reports HISTORY recovery response timing. Source-to-client benchmarks exclude browser rendering delay.

Run logs, per-run CSV/JSON records and the latest SUMMARY.md provide audit evidence. Policy comparisons in Section 6 used matched settings per mode and were not pooled with the separately reported T3/T5 runs. The supplied summary reports 3 runs per scenario and zero observed duplicates/stale messages in all 15 rows, but it does not embed a tested git revision. Future reruns should record `git rev-parse HEAD` and `git status --short` with machine configuration. Automated grading depends on the submitted code, not the report alone.

Sequential-source verification. An additional end-to-end integration test. The test maintained one server process and one client WebSocket connection while two sequential caption sources each transmitted three segments, restarting source-side segment numbering at 1. The server correctly normalized the segment IDs, producing finalized segments 1-3 in the first round and 4-6 in the second round.

The test passed, confirming that consecutive source runs were accepted without restarting the server or disconnecting the client, and that no duplicate segment identifiers were produced across the two rounds. The test output was retained as supplementary validation evidence.

```
python solution_server.py --port 8765      # terminal 1
python check_liverecall_repeat.py     # terminal 2
python -m unittest discover -s tests -t .     # regression checks
```

Separate monitoring evidence from a 100-client slow-reader run: five snapshots over 20 seconds measured 32.8–33.4 MiB RSS, 10 congested clients and 390 cumulative coalesced updates. This does not establish longer-run memory stabilisation.

Performance-result provenance. The T1–T5 performance measurements and FIFO/coalesce/adaptive comparisons were obtained from the archived Ubuntu experiments, with three runs per workload. The reported results were verified against the corresponding run records and summarised using the supplied analysis scripts. However, the archive does not contain an embedded tested-commit identifier. Therefore, these results are presented as historical experimental evidence and are not claimed to have been reproduced using the subsequent sequential-source implementation.

The repeat-source validation is a separate functional correctness test and does not establish the performance of the updated implementation under the T1–T5 workloads.

Appendix A. Server architecture and per-client delivery

A.5 LiveRecall server architecture

The diagram summarises the original server data path, plus the implemented normalisation of restarted source segment IDs at ingestion. Each accepted update enters authoritative session state before being offered to independent per-client channels. Source and browser remain separate from the server.

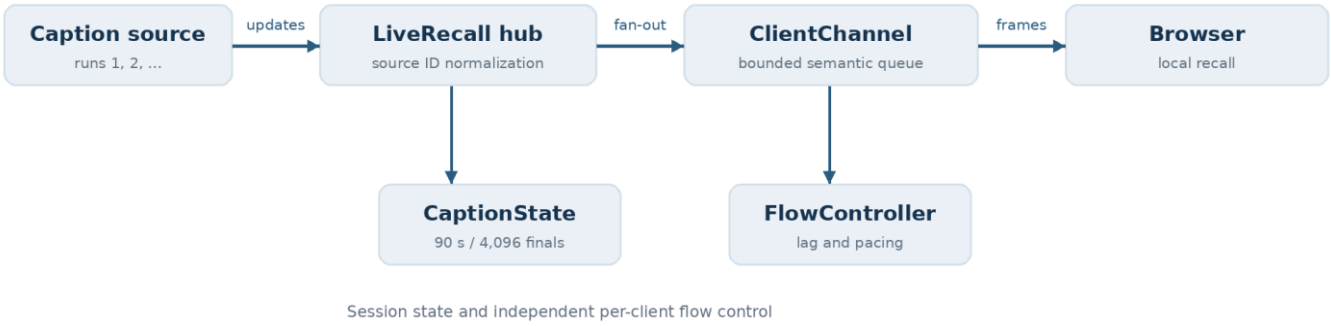


Figure A.1. caption_source.py → hub → session state / client channels → browser, FlowController governs individual writers and policy transitions. On join/reconnect, each client receives missing recent HISTORY and active state before continuing live delivery.

A.6 Multi-round segment sequence example

Source connection	Source-supplied segment IDs	Hub-visible IDs	Expected browser behaviour
Round 1	1, 2, 3	1, 2, 3	Render provisional-to-final replacement
Round 2	1, 2, 3	4, 5, 6	Continue existing transcript, no duplicates
Round 3	1, 2, 3	7, 8, 9	Continue, while old local history expires by age

Boundary condition. If the first source update does not begin at segment 1, the guarded patch does not infer a new numbering epoch. The submitted team should either confirm this assumption for its deterministic source or implement an explicit epoch mechanism before claiming more general support. With server restart, per-session highest-segment state is reset, persistence is outside the present assignment requirements.

IMPLEMENTATION / BENCHMARK DISTINCTION
The sequential-source regression validates functional correctness only. The archived Ubuntu T1–T5 benchmarks predate the extension and are not presented as performance measurements of the final implementation.